

Group Report: Snowflake Research Assistant Enhancement

Team Members: Blake Simpson, Rohan Ashraf Hashmi, Kenneth Kakie

Project: snowflake-research-assistant

1. Application Improvements

In this phase, we implemented several critical enhancements to transform our prototype into a production-ready application. Our focus was on improving visibility into the internal reasoning of our research assistant and providing a centralized interface for performance monitoring.

- **Integrated Monitoring Dashboard:** We added a dedicated dashboard tab to the Streamlit frontend. This allows us to toggle between the active "Chat" interface and a "Dashboard" view, providing real-time insights into how the system is performing without leaving the application.
- **Structured Logging Framework:** We replaced basic console print statements with a formal Python logging module. This improvement ensures that the backend captures the query lifecycle, warnings for fallbacks, and detailed error exceptions with standardized timestamps and query IDs.
- **Agent Tool Tracking:** To better understand the decision-making process of our agent, we implemented tracking for individual tool calls. The backend now records the sequence and count of calls made to the vector search and knowledge graph tools for every query. This data is returned in the API response and displayed in the frontend, providing transparency into the agent's reasoning steps.

2. Monitoring and Evaluation Features

We established a robust evaluation pipeline to ensure the system remains reliable and efficient.

- **Metrics Collection:** We implemented a synchronization process that logs detailed performance data into a dedicated Snowflake table (APP.EVAL_METRICS) for every request. The metrics collected include:
 - **Latency:** Measured in milliseconds to identify bottlenecks.
 - **Confidence Scores:** Capturing the model's self-reported certainty.
 - **Retrieval Modes:** Tracking whether the system utilized vector search, graph search, or a hybrid approach.
 - **Context Usage:** Recording the specific data chunks used to generate responses.
- **Visualizations:** The dashboard translates these logs into actionable insights through several visualizations:
 - **Summary Metrics:** High-level counters for total queries, average latency, and average confidence.

- **Latency Over Time:** A line chart visualizing performance trends across the last 24 hours and 7 days.
- **Tool Usage Breakdown:** A chart showing the distribution of calls between the Vector Database and Knowledge Graph.
- **Confidence Distribution:** A histogram to help us understand the reliability of the system's output across the entire corpus.

3. Deployment Method

Our deployment strategy was designed for stability and continuous integration using Railway.

- **Railway Setup:** We configured a multi-service architecture on Railway consisting of a FastAPI backend and a Streamlit frontend. We utilized railway.json and Procfile configurations to manage the environment and ensure the services scale appropriately.
- **Automated CI/CD:** We connected our GitHub repository directly to Railway with auto-deploy enabled. This allows any changes pushed to the main branch to be automatically built and redeployed, ensuring the live environment is always up to date.
- **Authentication Approach:** To handle Snowflake connectivity in a cloud environment, we moved all sensitive credentials to Railway environment variables. We also addressed Snowflake's Multi-Factor Authentication (MFA) requirements for deployed services by investigating key-pair authentication and disabling interactive MFA for the service account to prevent deployment interruptions.

4. Extension of the Phase-2 Prototype

This project significantly extends our Phase-2 prototype by moving it from a localized research tool to a monitored, cloud-deployed application.

While Phase-2 successfully demonstrated the core RAG (Retrieval-Augmented Generation) capabilities and Snowflake integration, it lacked the "Day 2" operational features required for a professional system. We have transitioned the backend from a "black box" that simply returned answers into a transparent system that logs every reasoning step and performance metric.

Furthermore, by moving from a local development environment to a production Railway deployment, we have resolved the complexities of cross-service communication and persistent authentication. The addition of the monitoring dashboard ensures that we can now evaluate the quality of our research assistant using data-driven metrics rather than anecdotal testing.

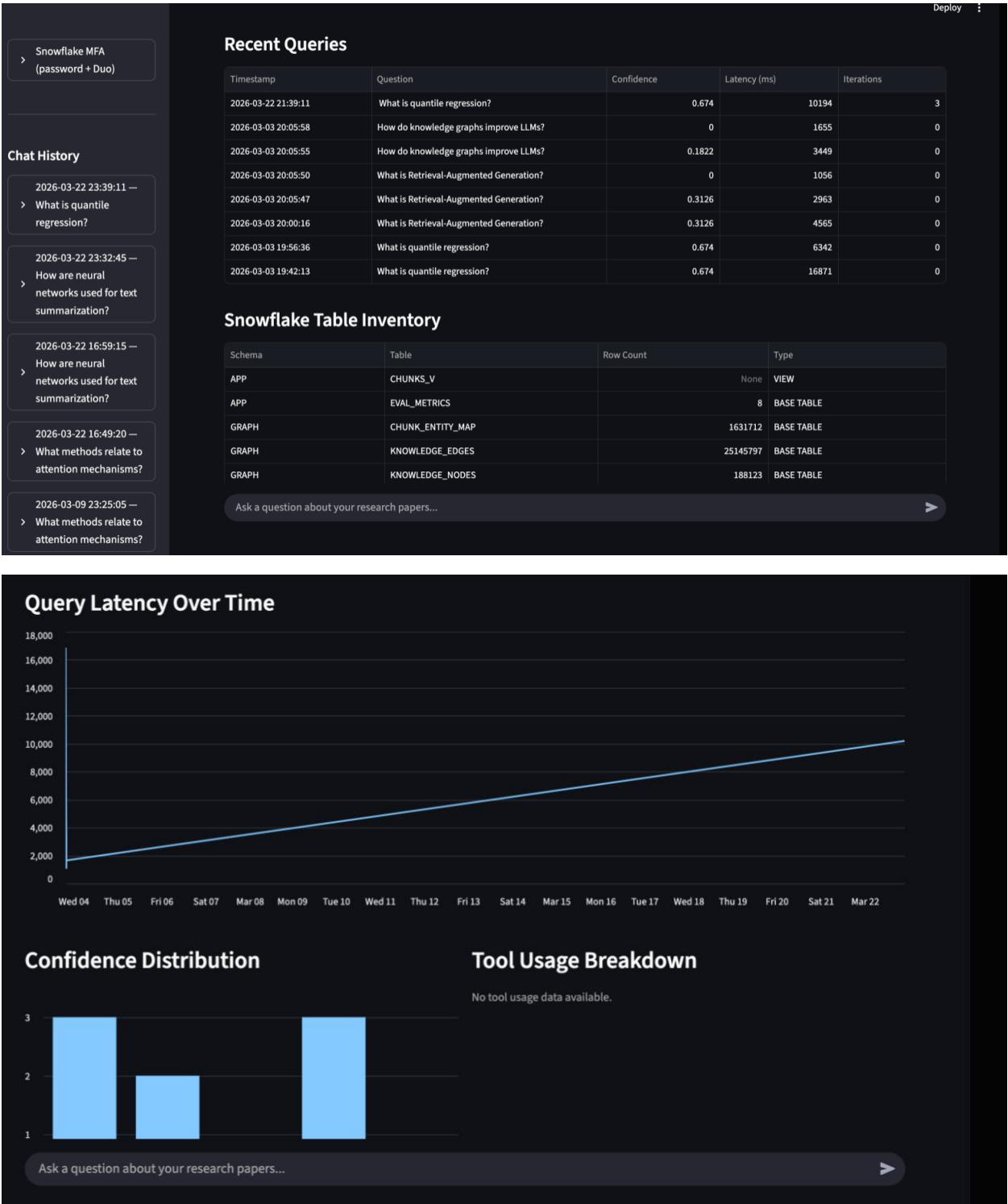
Project Links and Assets

GitHub Repository: <https://github.com/BigDataAnalytics-CS5542/snowflake-research-assistant>

Railway Deployment URLs:

- **Frontend:** <https://frontend-production-c6c11.up.railway.app/>
- **Backend:** <https://backend-production-b66e.up.railway.app/health>

Dashboard Screenshots:



Research Assistant

Personalized research assistant powered by Snowflake and Knowledge Graphs.

Chat **Dashboard**

Metrics Dashboard

Total Queries	Avg Latency	Avg Confidence	Papers in Corpus
N/A	N/A	N/A	1000

Query Latency Over Time



Ask a question about your research papers...

